

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology



Department of Computer Science & Engineering
24 Series

Course No.: CSE 1203

Course Title: Object Oriented Programmimg

Assignment Title : Train Tracker

<u>Submitted by:</u> Muhammad Ragib Ishrak Roll: 2403013 Section: A	<u>Submitted to:</u> Dr. Md. Shahid Uz Zaman Professor Dept. of CSE, RUET Shyla Afroge Associate Professor Dept. of CSE, RUET
---	--

Date of Submission : 29 April 2026

Problem Statement:

Write a Java Program that accepts the name/number of a train and displays its current location-based information.

Java Code:

```
import javax.swing.*;
import javax.swing.border.*;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.time.*;
import java.time.format.*;
import java.util.*;
import java.util.List;

// -----
// Interface
// -----
interface Trackable {
    String getCurrentStation(LocalTime now);
    int getStopsCompleted(LocalTime now);
    long getRemainingDistance(LocalTime now);
    long getRemainingMinutes(LocalTime now);
}

// -----
// Abstract class
// -----
abstract class RailwayEntity {
    protected String name;
    public RailwayEntity(String name) { this.name = name; }
    public String getName() { return name; }
    public abstract String getInfo();
}

// -----
// Station class
// -----
class Station extends RailwayEntity {
    private int distanceFromStart; // km
    private LocalTime arrival;
    private LocalTime departure;
```

```

    public Station(String name, int distanceFromStart, LocalTime arrival,
LocalTime departure) {
        super(name);
        this.distanceFromStart = distanceFromStart;
        this.arrival = arrival;
        this.departure = departure;
    }

    public int getDistanceFromStart() { return distanceFromStart; }
    public LocalTime getArrival()      { return arrival; }
    public LocalTime getDeparture()    { return departure; }

    @Override
    public String getInfo() {
        return name + " | Dist: " + distanceFromStart + " km | Arr: " + arrival +
" | Dep: " + departure;
    }
}

// -----
// Train class
// -----
class Train extends RailwayEntity implements Trackable, Serializable {
    private static final long serialVersionUID = 1L;

    private int    number;
    private List<Station> stations;
    private Set<DayOfWeek> offDays;
    private int    totalDistance;

    public Train(int number, String name, List<Station> stations,
        Set<DayOfWeek> offDays, int totalDistance) {
        super(name);
        this.number      = number;
        this.stations    = stations;
        this.offDays     = offDays;
        this.totalDistance = totalDistance;
    }

    public int          getNumber()      { return number; }
    public List<Station> getStations()    { return stations; }
    public Station      getStart()       { return stations.get(0); }
    public Station      getEnd()         { return stations.get(stations.size()
- 1); }
    public int          getTotalDistance(){ return totalDistance; }

```

```

public int      getStopCount()    { return stations.size(); }
public boolean  isOffDay(DayOfWeek d) { return offDays.contains(d); }
public Set<DayOfWeek> getOffDays() { return offDays; }

// — Trackable implementation —————
@Override
public String getCurrentStation(LocalTime now) {
    if (now.isBefore(getStart().getDeparture())) return getStart().getName();
    if (now.isAfter(getEnd().getArrival()))      return getEnd().getName();
    for (int i = 0; i < stations.size() - 1; i++) {
        Station cur  = stations.get(i);
        Station next = stations.get(i + 1);
        if (!now.isBefore(cur.getDeparture()) &&
now.isBefore(next.getArrival()))
            return "Between " + cur.getName() + " → " + next.getName();
        if (!now.isBefore(next.getArrival()) && (i + 1 == stations.size() - 1
|| now.isBefore(next.getDeparture()))
            return next.getName();
    }
    return getEnd().getName();
}

@Override
public int getStopsCompleted(LocalTime now) {
    int count = 0;
    for (Station s : stations) {
        if (!now.isBefore(s.getArrival())) count++;
        else break;
    }
    return Math.max(1, count);
}

@Override
public long getRemainingDistance(LocalTime now) {
    int distTravelled = 0;
    for (int i = 0; i < stations.size() - 1; i++) {
        Station cur  = stations.get(i);
        Station next = stations.get(i + 1);
        if (now.isAfter(cur.getDeparture()) &&
now.isBefore(next.getArrival())) {
            // interpolate
            long segDur  = java.time.Duration.between(cur.getDeparture(),
next.getArrival()).toMinutes();
            long elapsed = java.time.Duration.between(cur.getDeparture(),
now).toMinutes();

```

```

        int segDist = next.getDistanceFromStart() -
cur.getDistanceFromStart();
        distTravelled = cur.getDistanceFromStart() + (int)(segDist *
elapsed / (double) segDur);
        return totalDistance - distTravelled;
    }
}
// at a station
for (int i = stations.size() - 1; i >= 0; i--) {
    if (!now.isBefore(stations.get(i).getArrival())) {
        return totalDistance - stations.get(i).getDistanceFromStart();
    }
}
return totalDistance;
}

@Override
public long getRemainingMinutes(LocalTime now) {
    LocalTime arrival = getEnd().getArrival();
    if (now.isAfter(arrival)) return 0;
    return java.time.Duration.between(now, arrival).toMinutes();
}

public String getNextStation(LocalTime now) {
    for (int i = 0; i < stations.size() - 1; i++) {
        Station cur = stations.get(i);
        Station next = stations.get(i + 1);
        if (!now.isBefore(cur.getDeparture()) &&
now.isBefore(next.getArrival()))
            return next.getName();
    }
    return getEnd().getName();
}

public String getNextStop(LocalTime now) {
    for (int i = 0; i < stations.size() - 1; i++) {
        if (now.isBefore(stations.get(i + 1).getArrival()))
            return stations.get(i + 1).getName();
    }
    return getEnd().getName();
}

@Override
public String getInfo() {
    return number + "-" + name;
}

```

```

    }
}

// -----
// Data Store (permanent save / load via serialization)
// -----
class TrainDataStore {
    private static final String FILE = "train_data.ser";

    public static void save(List<Train> trains) {
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream(FILE))) {
            oos.writeObject(trains);
        } catch (Exception e) {
            // silent - file will be recreated next run
        }
    }

    @SuppressWarnings("unchecked")
    public static List<Train> load() {
        File f = new File(FILE);
        if (!f.exists()) return null;
        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream(f))) {
            return (List<Train>) ois.readObject();
        } catch (Exception e) {
            return null;
        }
    }
}

// -----
// Progress Bar Panel (custom painted)
// -----
class StopProgressBar extends JPanel {
    private int          total, completed;
    private List<Station> stations;
    private int          hoveredIndex = -1;

    // — Popup panel -----
    private JWindow popup;
    private JLabel  popName, popArr, popDep, popDist, popStatus;

    public StopProgressBar() {
        setOpaque(false);
    }
}

```

```

        setPreferredSize(new Dimension(400, 28));
        setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
        buildPopup();
        wireMouseListeners();
    }

    private void buildPopup() {
        popup = new JWindow();
        popup.setAlwaysOnTop(true);

        JPanel card = new JPanel();
        card.setLayout(new BoxLayout(card, BoxLayout.Y_AXIS));
        card.setBackground(new Color(20, 23, 34));
        card.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(new Color(0, 200, 200), 1),
            BorderFactory.createEmptyBorder(8, 12, 8, 12)
        ));

        Font boldF = new Font("Segoe UI", Font.BOLD, 12);
        Font plainF = new Font("Segoe UI", Font.PLAIN, 11);

        popName = makePopLabel("", boldF, new Color(255, 200, 0));
        popArr = makePopLabel("", plainF, Color.WHITE);
        popDep = makePopLabel("", plainF, Color.WHITE);
        popDist = makePopLabel("", plainF, new Color(150, 220, 255));
        popStatus = makePopLabel("", boldF, new Color(0, 220, 120));

        card.add(popName);
        card.add(Box.createVerticalStrut(4));
        card.add(popArr);
        card.add(popDep);
        card.add(popDist);
        card.add(Box.createVerticalStrut(4));
        card.add(popStatus);

        popup.getContentPane().add(card);
    }

    private JLabel makePopLabel(String text, Font font, Color color) {
        JLabel l = new JLabel(text);
        l.setFont(font);
        l.setForeground(color);
        l.setAlignmentX(Component.LEFT_ALIGNMENT);
        return l;
    }
}

```

```

private void wireMouseListeners() {
    addMouseListener(new java.awt.event.MouseMotionAdapter() {
        @Override
        public void mouseMoved(java.awt.event.MouseEvent e) {
            int idx = hitIndex(e.getX(), e.getY());
            if (idx != hoveredIndex) {
                hoveredIndex = idx;
                repaint();
                if (idx >= 0 && stations != null && idx < stations.size()) {
                    showPopup(e, idx);
                } else {
                    popup.setVisible(false);
                }
            }
        }
    });

    addMouseListener(new java.awt.event.MouseAdapter() {
        @Override
        public void mouseClicked(java.awt.event.MouseEvent e) {
            int idx = hitIndex(e.getX(), e.getY());
            if (idx >= 0 && stations != null && idx < stations.size()) {
                showPopup(e, idx);
            }
        }
        @Override
        public void mouseExited(java.awt.event.MouseEvent e) {
            hoveredIndex = -1;
            repaint();
            popup.setVisible(false);
        }
    });
}

```

```

/** Returns dot index under (mx, my), or -1 if none. */
private int hitIndex(int mx, int my) {
    if (total == 0) return -1;
    int dotD    = 14, dotY = getHeight() / 2;
    int spacing = (getWidth() - 20) / Math.max(total - 1, 1);
    int hitR    = dotD; // slightly larger hit area
    for (int i = 0; i < total; i++) {
        int cx = 10 + spacing * i;
        int dx = mx - cx, dy = my - dotY;
        if (dx * dx + dy * dy <= hitR * hitR) return i;
    }
}

```



```

    }
    return -1;
}

private void showPopup(java.awt.event.MouseEvent e, int idx) {
    Station s = stations.get(idx);
    boolean done = idx < completed;
    boolean cur = idx == completed - 1;

    popName .setText("🚉 " + s.getName());
    popArr .setText(" Arrival : " +
s.getArrival() .format(java.time.format.DateTimeFormatter.ofPattern("HH:mm")));
    popDep .setText(" Departure: " +
s.getDeparture().format(java.time.format.DateTimeFormatter.ofPattern("HH:mm")));
    popDist .setText(" Distance : " + s.getDistanceFromStart() + " km from
start");

    if (cur) {
        popStatus.setForeground(new Color(255, 200, 0));
        popStatus.setText(" ▶ Current Position");
    } else if (done) {
        popStatus.setForeground(new Color(0, 200, 100));
        popStatus.setText(" ✓ Already Passed");
    } else {
        popStatus.setForeground(new Color(220, 80, 80));
        popStatus.setText(" ● Yet to Reach");
    }

    popup.pack();
    // position popup just above the dot
    Point screen = e.getComponent().getLocationOnScreen();
    int px = screen.x + e.getX() - popup.getWidth() / 2;
    int py = screen.y - popup.getHeight() - 6;
    popup.setLocation(px, py);
    popup.setVisible(true);
}

public void update(int total, int completed, List<Station> stations) {
    this.total = total;
    this.completed = completed;
    this.stations = stations;
    repaint();
}

// keep old 2-arg overload for safety

```

```

public void update(int total, int completed) {
    update(total, completed, this.stations);
}

@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    if (total == 0) return;
    Graphics2D g2 = (Graphics2D) g;
    g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
        RenderingHints.VALUE_ANTIALIAS_ON);

    int w = getWidth(), h = getHeight();
    int dotD = 14, dotY = h / 2;
    int spacing = (w - 20) / Math.max(total - 1, 1);

    // yellow line - travelled
    if (completed > 1) {
        int x1 = 10, x2 = 10 + spacing * (completed - 1);
        g2.setColor(new Color(255, 200, 0));
        g2.setStroke(new BasicStroke(4));
        g2.drawLine(x1, dotY, x2, dotY);
    }
    // grey line - remaining
    if (completed < total) {
        int x1 = 10 + spacing * Math.max(completed - 1, 0);
        int x2 = 10 + spacing * (total - 1);
        g2.setColor(Color.GRAY);
        g2.setStroke(new BasicStroke(2));
        g2.drawLine(x1, dotY, x2, dotY);
    }

    // dots
    for (int i = 0; i < total; i++) {
        int cx = 10 + spacing * i;
        boolean hovered = (i == hoveredIndex);
        int r = hovered ? dotD + 4 : dotD; // enlarge on hover

        g2.setColor(i < completed ? new Color(0, 180, 0) : new Color(200, 40,
40));

        g2.fillOval(cx - r / 2, dotY - r / 2, r, r);

        // white ring; gold ring when hovered
        g2.setColor(hovered ? new Color(255, 200, 0) : Color.WHITE);
        g2.setStroke(new BasicStroke(hovered ? 2.5f : 1.5f));
    }
}

```

```

        g2.drawOval(cx - r / 2, dotY - r / 2, r, r);
    }
}

// -----
// Main GUI
// -----
public class Assignment_CSE1203 extends JFrame {

    private List<Train>      trains;
    private JComboBox<String> combo;
    private JLabel[]        valueLabels = new JLabel[10];
    private StopProgressBar progressBar;
    private JLabel          statusLabel;
    private JLabel          clockLabel;
    private javax.swing.Timer timer;
    private javax.swing.Timer clockTimer;

    // — Color palette —
    private static final Color BG_DARK    = new Color(35, 38, 48);
    private static final Color BG_PANEL   = new Color(22, 25, 35);
    private static final Color ACCENT     = new Color(160, 30, 30);
    private static final Color GOLD       = new Color(255, 200, 0);
    private static final Color TEXT_WHITE = Color.WHITE;
    private static final Color TEXT_CYAN  = new Color(0, 220, 220);

    public Assignment_CSE1203() {
        trains = buildTrains();
        TrainDataStore.save(trains);          // save permanently

        setTitle("Bangladesh Train Tracker");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setSize(820, 430);
        setLocationRelativeTo(null);
        setResizable(false);

        buildUI();

        // auto-refresh train info every 30 seconds
        timer = new javax.swing.Timer(30_000, e -> refreshInfo());
        timer.start();

        // real-time clock – ticks every second
        clockTimer = new javax.swing.Timer(1_000, e -> tickClock());

```

```

        clockTimer.start();

        setVisible(true);
        refreshInfo();
        tickClock();
    }

    // — UI construction —————
    private void buildUI() {
        JPanel root = new JPanel(new BorderLayout());
        root.setBackground(BG_DARK);
        setContentPane(root);

        // — Title bar —————
        JPanel titleBar = new JPanel(new BorderLayout());
        titleBar.setBackground(ACCENT);
        titleBar.setBorder(new EmptyBorder(6, 14, 6, 14));
        JLabel title = new JLabel("Bangladesh Train Tracker",
SwingConstants.CENTER);
        title.setForeground(TEXT_WHITE);
        title.setFont(new Font("Segoe UI", Font.BOLD, 16));

        // Real-time clock in title bar
        clockLabel = new JLabel("", SwingConstants.RIGHT);
        clockLabel.setForeground(GOLD);
        clockLabel.setFont(new Font("Segoe UI", Font.BOLD, 13));

        titleBar.add(title, BorderLayout.CENTER);
        titleBar.add(clockLabel, BorderLayout.EAST);
        root.add(titleBar, BorderLayout.NORTH);

        // — Left panel —————
        JPanel left = new JPanel();
        left.setLayout(new BoxLayout(left, BoxLayout.Y_AXIS));
        left.setBackground(new Color(45, 50, 65));
        left.setBorder(new EmptyBorder(20, 16, 10, 16));
        left.setPreferredSize(new Dimension(210, 0));

        JLabel selLabel = new JLabel("Select Train");
        selLabel.setForeground(TEXT_WHITE);
        selLabel.setFont(new Font("Segoe UI", Font.PLAIN, 13));
        selLabel.setAlignmentX(Component.LEFT_ALIGNMENT);

        String[] names =
trains.stream().map(Train::getInfo).toArray(String[]::new);

```

```

combo = new JComboBox<>(names);
combo.setMaximumSize(new Dimension(180, 28));
combo.setAlignmentX(Component.LEFT_ALIGNMENT);
combo.setFont(new Font("Segoe UI", Font.PLAIN, 12));
combo.addActionListener(e -> refreshInfo());

// Load Bangladesh Railway logo
JLabel logo = new JLabel();
logo.setAlignmentX(Component.CENTER_ALIGNMENT);
try {
    File imgFile = new File("br_logo.png");
    if (imgFile.exists()) {
        ImageIcon icon = new ImageIcon(imgFile.getAbsolutePath());
        Image img = icon.getImage().getScaledInstance(160, 100,
Image.SCALE_SMOOTH);
        logo.setIcon(new ImageIcon(img));
    } else {
        // fallback: try classpath
        java.net.URL imgUrl = getClass().getResource("/br_logo.png");
        if (imgUrl != null) {
            ImageIcon icon = new ImageIcon(imgUrl);
            Image img = icon.getImage().getScaledInstance(160, 100,
Image.SCALE_SMOOTH);
            logo.setIcon(new ImageIcon(img));
        } else {
            logo.setText("🚂");
            logo.setFont(new Font("Segoe UI Emoji", Font.PLAIN, 48));
            logo.setForeground(GOLD);
        }
    }
} catch (Exception ex) {
    logo.setText("🚂");
    logo.setFont(new Font("Segoe UI Emoji", Font.PLAIN, 48));
    logo.setForeground(GOLD);
}

left.add(sellLabel);
left.add(Box.createVerticalStrut(6));
left.add(combo);
left.add(Box.createVerticalStrut(20));
left.add(logo);
left.add(Box.createVerticalGlue());

JLabel copy = new JLabel("Copyright@2026, CSE RUET");
copy.setForeground(new Color(150, 150, 150));

```

```

copy.setFont(new Font("Segoe UI", Font.PLAIN, 10));
copy.setAlignmentX(Component.LEFT_ALIGNMENT);
left.add(copy);

root.add(left, BorderLayout.WEST);

// — Right info panel —
JPanel right = new JPanel(new BorderLayout());
right.setBackground(BG_PANEL);
right.setBorder(new CompoundBorder(
    new MatteBorder(0, 2, 0, 0, ACCENT),
    new EmptyBorder(12, 18, 10, 18)));

// Info grid
JPanel grid = new JPanel(new GridLayout(10, 2, 4, 3));
grid.setBackground(BG_PANEL);

String[] keys = {
    "1. Train Number:", "2. Train Name:",
    "3. Start Station:", "4. Time of Departure:",
    "5. Destination Station:", "6. Time of Arrival:",
    "7. Number of Stops:", "8. Next Station:",
    "9. Next Stop:", "10. Total Distance:"
};

Font keyFont = new Font("Segoe UI", Font.PLAIN, 13);
Font valFont = new Font("Segoe UI", Font.BOLD, 13);

for (int i = 0; i < keys.length; i++) {
    JLabel k = new JLabel(keys[i]);
    k.setForeground(TEXT_WHITE);
    k.setFont(keyFont);
    grid.add(k);

    valueLabels[i] = new JLabel("-");
    valueLabels[i].setForeground(TEXT_CYAN);
    valueLabels[i].setFont(valFont);
    grid.add(valueLabels[i]);
}

right.add(grid, BorderLayout.CENTER);

// Progress + status at bottom
JPanel bottom = new JPanel(new BorderLayout(6, 0));
bottom.setBackground(BG_PANEL);

```

```

        bottom.setBorder(new EmptyBorder(8, 0, 0, 0));

        progressBar = new StopProgressBar();
        statusLabel = new JLabel("", SwingConstants.CENTER);
        statusLabel.setForeground(GOLD);
        statusLabel.setFont(new Font("Segoe UI", Font.BOLD, 12));

        bottom.add(progressBar, BorderLayout.CENTER);
        bottom.add(statusLabel, BorderLayout.SOUTH);

        right.add(bottom, BorderLayout.SOUTH);
        root.add(right, BorderLayout.CENTER);
    }

    // — Real-time clock tick (every second) —
    private void tickClock() {
        LocalDateTime now = LocalDateTime.now();
        String timeStr = now.format(DateTimeFormatter.ofPattern("HH:mm:ss"));
        String dateStr = now.format(DateTimeFormatter.ofPattern("EEE, dd MMM
yyyy"));
        clockLabel.setText(dateStr + " | " + timeStr + " ");
        // Also refresh remaining time display every second
        updateStatusOnly();
    }

    // — Update only the status/remaining line (lightweight) —
    private void updateStatusOnly() {
        int idx = combo.getSelectedIndex();
        if (idx < 0) return;
        Train t = trains.get(idx);
        LocalTime now = LocalTime.now();
        DayOfWeek dow = LocalDate.now().getDayOfWeek();

        if (t.isOffDay(dow)) return;

        long remDist = t.getRemainingDistance(now);
        long remMins = t.getRemainingMinutes(now);
        long remSecs = java.time.Duration.between(now,
t.getEnd().getArrival()).toSeconds();

        // Update next station live
        valueLabels[7].setText(t.getCurrentStation(now));
        valueLabels[8].setText(t.getNextStop(now));

        // Update progress bar stops

```

```

        progressBar.update(t.getStopCount(), t.getStopsCompleted(now),
t.getStations());

        if (now.isAfter(t.getEnd().getArrival())) {
            statusLabel.setText("✓ Train has arrived at " +
t.getEnd().getName());
        } else if (now.isBefore(t.getStart().getDeparture())) {
            long secsToDepart = java.time.Duration.between(now,
t.getStart().getDeparture()).toSeconds();
            long dh = secsToDepart / 3600, dm = (secsToDepart % 3600) / 60, ds =
secsToDepart % 60;
            statusLabel.setText(String.format("Departs in %02d:%02d:%02d", dh,
dm, ds));
        } else {
            long hh = remSecs / 3600, mm = (remSecs % 3600) / 60, ss = remSecs %
60;

            statusLabel.setText(String.format(
                "Remaining %d km | %02d:%02d:%02d left", remDist, hh, mm,
ss));
        }
    }

    private void refreshInfo() {
        int idx = combo.getSelectedIndex();
        if (idx < 0) return;
        Train t = trains.get(idx);
        LocalTime now = LocalTime.now();
        DayOfWeek dow = LocalDate.now().getDayOfWeek();

        // Off-day check
        if (t.isOffDay(dow)) {
            String offMsg = "Train " + t.getInfo() + " does NOT operate on " +
dow;

            for (JLabel v : valueLabels) v.setText("-");
            statusLabel.setText(offMsg);
            progressBar.update(0, 0);
            return;
        }

        int stopsTotal = t.getStopCount();
        int stopsCompleted = t.getStopsCompleted(now);
        long remDist = t.getRemainingDistance(now);
        long remMins = t.getRemainingMinutes(now);

```



```

        valueLabels[0].setText(String.valueOf(t.getNumber()));
        valueLabels[1].setText(t.getName());
        valueLabels[2].setText(t.getStart().getName());
        valueLabels[3].setText(t.getStart().getDeparture().format(DateTimeFormatt
er.ofPattern("HH:mm"))));
        valueLabels[4].setText(t.getEnd().getName());
        valueLabels[5].setText(t.getEnd().getArrival().format(DateTimeFormatter.o
fPattern("HH:mm"))));
        valueLabels[6].setText(stopsTotal + " stops");
        valueLabels[7].setText(t.getCurrentStation(now));
        valueLabels[8].setText(t.getNextStop(now));
        valueLabels[9].setText(t.getTotalDistance() + " km");

        progressBar.update(stopsTotal, stopsCompleted, t.getStations());

        if (now.isAfter(t.getEnd().getArrival())) {
            statusLabel.setText("Train has arrived at " + t.getEnd().getName());
        } else if (now.isBefore(t.getStart().getDeparture())) {
            statusLabel.setText("Train departs at " +
t.getStart().getDeparture());
        } else {
            long hh = remMins / 60, mm = remMins % 60;
            statusLabel.setText(String.format(
                "Remaining %d km in %02d:%02d hours", remDist, hh, mm));
        }
    }

    // — Train data (based on Bangladesh Railway) —
    private List<Train> buildTrains() {
        List<Train> list = new ArrayList<>();

        // — 754 Silkcity Express (Rajshahi → Dhaka) —
        list.add(new Train(754, "Silkcity Express",
            Arrays.asList(
                new Station("Rajshahi", 0, LocalTime.of( 7,35), LocalTime.of(
7,40)),
                new Station("Abhaynagar",20, LocalTime.of( 8, 5), LocalTime.of(
8, 7)),
                new Station("Natore", 45, LocalTime.of( 8,40), LocalTime.of(
8,43)),
                new Station("Baraigram", 62, LocalTime.of( 9, 3), LocalTime.of(
9, 6)),
                new Station("Ullapara", 80, LocalTime.of( 9,28), LocalTime.of(
9,30)),

```

```

        new Station("Jamtail", 100, LocalTime.of( 9,53), LocalTime.of(
9,56)),
        new Station("Solop", 118, LocalTime.of(10,18),
LocalTime.of(10,20)),
        new Station("Sirajganj",135, LocalTime.of(10,42),
LocalTime.of(10,45)),
        new Station("Bhuapur", 158, LocalTime.of(11,12),
LocalTime.of(11,15)),
        new Station("Tangail", 190, LocalTime.of(11,52),
LocalTime.of(11,55)),
        new Station("Joydebpur",225, LocalTime.of(12,38),
LocalTime.of(12,40)),
        new Station("Dhaka", 255, LocalTime.of(13,20),
LocalTime.of(13,20))
    ),
    EnumSet.noneOf(DayOfWeek.class), // runs every day
    255
));

// — 725 Padma Express (Rajshahi → Dhaka) —
list.add(new Train(725, "Padma Express",
    Arrays.asList(
        new Station("Rajshahi", 0, LocalTime.of(14, 0),
LocalTime.of(14, 5)),
        new Station("Natore", 45, LocalTime.of(15, 0),
LocalTime.of(15, 3)),
        new Station("Ishwardi", 72, LocalTime.of(15,40),
LocalTime.of(15,45)),
        new Station("Pakshi", 80, LocalTime.of(15,58),
LocalTime.of(16, 0)),
        new Station("Poradaha", 110, LocalTime.of(16,38),
LocalTime.of(16,40)),
        new Station("Mirpur", 140, LocalTime.of(17,18),
LocalTime.of(17,20)),
        new Station("Jhenaidah",165, LocalTime.of(17,52),
LocalTime.of(17,55)),
        new Station("Faridpur", 200, LocalTime.of(18,45),
LocalTime.of(18,48)),
        new Station("Dhaka", 262, LocalTime.of(20, 0),
LocalTime.of(20, 0))
    ),
    EnumSet.of(DayOfWeek.FRIDAY), // off on Friday
    262
));

```

```

// — 791 Kapotaksha Express (Khulna → Rajshahi) —
list.add(new Train(791, "Kapotaksha Express",
    Arrays.asList(
        new Station("Khulna",      0,    LocalTime.of( 6, 0)),
LocalTime.of( 6, 5)),
        new Station("Jessore",    60,    LocalTime.of( 7,15)),
LocalTime.of( 7,18)),
        new Station("Chuadanga", 110,    LocalTime.of( 8,20)),
LocalTime.of( 8,23)),
        new Station("Poradaha",  145,    LocalTime.of( 9, 5)),
LocalTime.of( 9, 8)),
        new Station("Ishwardi",  185,    LocalTime.of( 9,58)),
LocalTime.of(10, 3)),
        new Station("Natore",    225,    LocalTime.of(10,52)),
LocalTime.of(10,55)),
        new Station("Abhaynagar",245,    LocalTime.of(11,22)),
LocalTime.of(11,24)),
        new Station("Rajshahi",  270,    LocalTime.of(12, 0)),
LocalTime.of(12, 0))
    ),
    EnumSet.of(DayOfWeek.TUESDAY),
    270
));

// — 725 Sundarban Express (Khulna → Dhaka) —
list.add(new Train(725, "Sundarban Express",
    Arrays.asList(
        new Station("Khulna",      0,    LocalTime.of( 6,20)),
LocalTime.of( 6,25)),
        new Station("Jessore",    60,    LocalTime.of( 7,35)),
LocalTime.of( 7,38)),
        new Station("Kotchandpur", 90,    LocalTime.of( 8,12)),
LocalTime.of( 8,15)),
        new Station("Poradaha",   125,    LocalTime.of( 8,58)),
LocalTime.of( 9, 1)),
        new Station("Chuadanga",  155,    LocalTime.of( 9,35)),
LocalTime.of( 9,38)),
        new Station("Ishwardi",   200,    LocalTime.of(10,28)),
LocalTime.of(10,33)),
        new Station("Ullapara",   225,    LocalTime.of(11, 2)),
LocalTime.of(11, 5)),
        new Station("Jamtoil",    248,    LocalTime.of(11,32)),
LocalTime.of(11,35)),
        new Station("Tangail",    310,    LocalTime.of(12,45)),
LocalTime.of(12,48)),
    )
));

```

```

        new Station("Joydebpur", 348, LocalTime.of(13,30),
LocalTime.of(13,32)),
        new Station("Dhaka", 390, LocalTime.of(14,25),
LocalTime.of(14,25))
    ),
    EnumSet.of(DayOfWeek.WEDNESDAY),
    390
));

// — 793 Bonolata Express (Dhaka → Rajshahi) —
list.add(new Train(793, "Bonolata Express",
    Arrays.asList(
        new Station("Dhaka", 0, LocalTime.of(14, 0),
LocalTime.of(14, 5)),
        new Station("Joydebpur", 30, LocalTime.of(14,48),
LocalTime.of(14,51)),
        new Station("Tangail", 68, LocalTime.of(15,40),
LocalTime.of(15,43)),
        new Station("Bangabandhu Bridge East",110, LocalTime.of(16,32),
LocalTime.of(16,37)),
        new Station("Sirajganj", 130, LocalTime.of(17, 5),
LocalTime.of(17, 8)),
        new Station("Ullapara", 148, LocalTime.of(17,32),
LocalTime.of(17,35)),
        new Station("Baraigram", 168, LocalTime.of(17,58),
LocalTime.of(18, 1)),
        new Station("Natore", 185, LocalTime.of(18,25),
LocalTime.of(18,28)),
        new Station("Abhaynagar", 210, LocalTime.of(19, 2),
LocalTime.of(19, 4)),
        new Station("Rajshahi", 230, LocalTime.of(19,40),
LocalTime.of(19,40))
    ),
    EnumSet.of(DayOfWeek.THURSDAY), // off on Thursday
    230
));

return list;
}

// — Entry point —————
public static void main(String[] args) {
    // Use system look and feel where possible
    try {
        UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName()); }

```

```

        catch (Exception ignored) {}

        SwingUtilities.invokeLater(Assignment_CSE1203::new);
    }
}

```

Output:

The screenshot shows the Bangladesh Train Tracker application window. The title bar reads "Bangladesh Train Tracker". The header bar is red and contains the application name and the date/time: "Tue, 28 Apr 2026 | 23:52:42". The main content area is dark blue. On the left, there is a "Select Train" dropdown menu with "754-Silkcity Express" selected. Below it is a yellow train icon. The right side of the window displays the following train details:

1. Train Number:	754
2. Train Name:	Silkcity Express
3. Start Station:	Rajshahi
4. Time of Departure:	07:40
5. Destination Station:	Dhaka
6. Time of Arrival:	13:20
7. Number of Stops:	12 stops
8. Next Station:	Dhaka
9. Next Stop:	Dhaka
10. Total Distance:	255 km

Below the details is a yellow line with green dots representing the route. At the bottom, a status bar indicates "Train has arrived at Dhaka".

The screenshot shows the Bangladesh Train Tracker application window. The title bar reads "Bangladesh Train Tracker". The header bar is red and contains the application name and the date/time: "Tue, 28 Apr 2026 | 23:54:40". The main content area is dark blue. On the left, there is a "Select Train" dropdown menu with "754-Silkcity Express" selected. Below it is a yellow train icon. The right side of the window displays the following train details:

1. Train Number:	754
2. Train Name:	Silkcity Express
3. Start Station:	Rajshahi
4. Time of Departure:	07:40
5. Destination Station:	Dhaka
6. Time of Arrival:	13:20
7. Number of Stops:	12 stops
8. Next Station:	Dhaka
9. Next Stop:	Dhaka
10. Total Distance:	255 km

Below the details is a yellow line with green dots representing the route. A pop-up window is displayed over the route, showing the following information:

Ullapara
Arrival : 09:28
Departure: 09:30
Distance : 80 km from start
Already Passed

At the bottom, a status bar indicates "Train has arrived at Dhaka".